Day 9 Master Summary: Hierarchical Clustering & DBSCAN

Author / Student: abs6187	**Curriculum Track:** Unsupervised Learning	**Date:** October 2026
Email: 23f2000876@ds.study.iitm.ac.in	**Topics:** Agglomerative, Dendrograms, Metrics, DBSCAN, KneeLocator	**Status:** Completed & Verified

1. Executive Overview: Unsupervised Clustering Taxonomy

Clustering partitions unlabeled data into natural geometric groupings without target labels. While **Day 8** addressed centroid partitioning via K-Means (which assumes convex spherical geometries and requires a pre-determined cluster count k), **Day 9** expands into two modern clustering paradigms:

- **Hierarchical Clustering (Agglomerative)**: A deterministic, bottom-up tree-building approach that builds an entire multi-scale cluster hierarchy without pre-specifying k. Clusters are defined by slicing a dendrogram tree at an optimal distance threshold.
- **Density-Based Spatial Clustering (DBSCAN)**: A non-parametric density connectivity algorithm capable of discovering arbitrary geometric topologies (crescents, rings, dense filaments) while explicitly detecting and isolating sparse anomaly/noise points (label -1).

Clustering Algorithm	Underlying Assumption	Geometry Handled	Sensitivity to Outliers	Time Complexity
K-Means	Voronoi cells around centroids	Spherical, convex only	High (drags centroid position)	O(n * k * d * iter)
Agglomerative Hierarchical	Pairwise linkage distance tree	Spherical (Ward) to Arbitrary (Single)	Moderate (isolated in late merges)	O(n^2 log n) to O(n^3)
DBSCAN	Local neighborhood sample density	Arbitrary, non-convex, manifold	Immune (isolated as noise label -1)	O(n log n) with spatial index

2. Agglomerative Hierarchical Clustering Mechanics

Agglomerative clustering begins with N singleton clusters C_i = {x_i}. At each step, the algorithm locates the closest pair of clusters according to a chosen **Linkage Criterion** and merges them: C_new = C_a U C_b. This process repeats N - 1 times until all points merge into a root node.

Linkage Method	Mathematical Distance Formula	Behavior & Structural Characteristics
Ward (linkage='ward')	d(A, B) = sqrt[(2|A||B| / (|A| + |B|)) * ||mu_A - mu_B||^2]	Minimizes total within-cluster variance (error sum of squares). Produces highly balanced, spherical clusters. Requires Euclidean metric.
Complete (linkage='complete')	d(A, B) = max { ||u - v|| : u in A, v in B }	Maximum pairwise distance between cluster members. Prevents stringy chaining, creates compact groups with equal diameter, but sensitive to outliers.
Average (UPGMA) (linkage='average')	d(A, B) = (1 / (|A||B|)) * sum_{u in A} sum_{v in B} ||u - v||	Mean pairwise distance across all member combinations. Highly balanced compromise between single and complete linkage.
Single (linkage='single')	d(A, B) = min { ||u - v|| : u in A, v in B }	Minimum pairwise distance. Capable of tracking non-linear concentric shapes, but severely susceptible to *chaining* (spurious bridges).

Cophenetic Correlation Coefficient (r)

To objectively validate which linkage criterion best preserves the original geometry, compute the **Cophenetic Correlation Coefficient**. The cophenetic distance c_ij is the height in the dendrogram where sample i and sample j first unite. The coefficient r is the Pearson correlation between original Euclidean distances d_ij and dendrogram merge heights c_ij:

r = sum[(d_ij - d_mean)(c_ij - c_mean)] / sqrt[sum((d_ij - d_mean)^2) * sum((c_ij - c_mean)^2)]

A correlation r >= 0.75 indicates high fidelity, confirming that cutting the tree at vertical height threshold t accurately reflects true pairwise relationships.

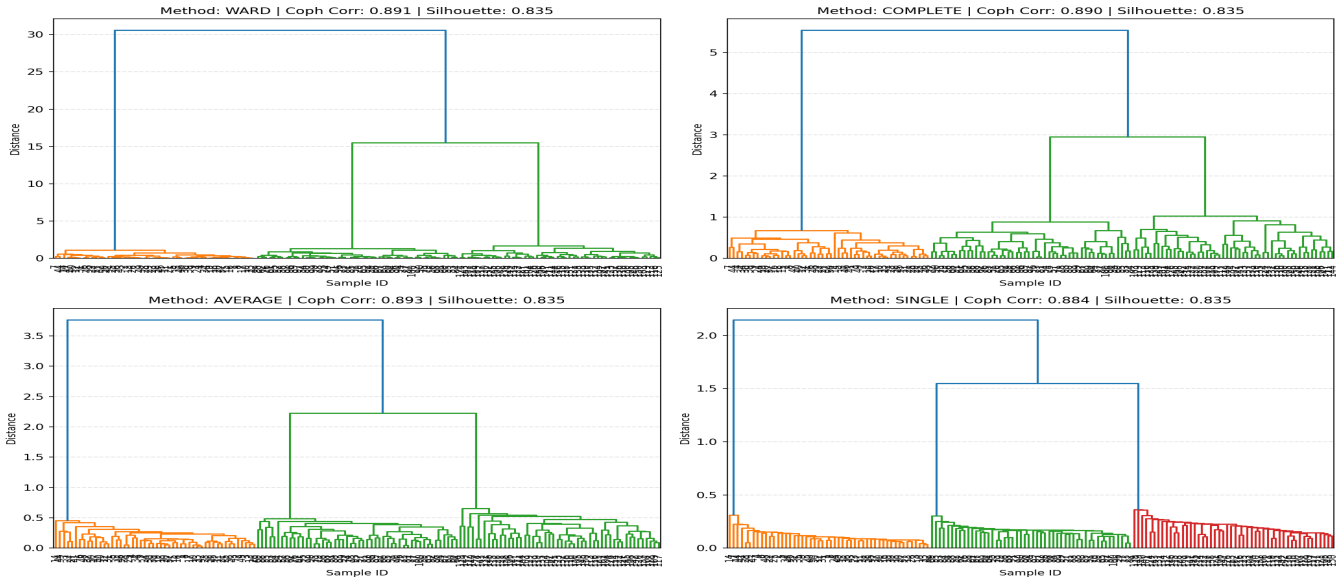


Figure 1: 4-Way Linkage Comparison across Ward, Complete, Average, and Single methods (Day9/01_exercise_linkage_comparison.ipynb).

3. Unsupervised Cluster Validation Metrics (DBI, Silhouette, CHI)

Without ground-truth target labels (y), standard classification metrics (Accuracy, ROC-AUC) cannot be computed. Instead, internal validation metrics quantify the mathematical balance between **Intra-Cluster Cohesion** (compactness) and **Inter-Cluster Separation** (distance between clusters):

Metric Name	Formula & Formulation	Optimal Criterion	Interpretation & Nuances
Davies-Bouldin Index (DBI)	DB = (1/k) * sum_{i=1}^k max_{j != i} [(s_i + s_j) / d(mu_i, mu_j)] Where s_i is cluster dispersion, d(mu_i, mu_j) is centroid distance.	**Lower is Better (Minimum >= 0)**	Evaluates the worst-case similarity between each cluster and its nearest neighbor. Penalizes broad, overlapping clusters.
Silhouette Score (s)	s(i) = [b(i) - a(i)] / max(a(i), b(i)) a(i) = mean intra-cluster distance b(i) = mean distance to closest neighbor cluster.	**Higher is Better Range: [-1.0, +1.0]**	Scores > 0.5 indicate strong, well-separated cluster membership; near 0 indicates overlapping boundaries; negative values denote sample misclassification.
Calinski-Harabasz Index (CHI)	CH = [Tr(B_k) / Tr(W_k)] * [(N - k) / (k - 1)] Ratio of between-cluster variance B_k to within-cluster variance W_k.	**Higher is Better (Peak values)**	Variance Ratio Criterion. Computationally fast (O(n)). Tends to favor convex, spherical clusters.

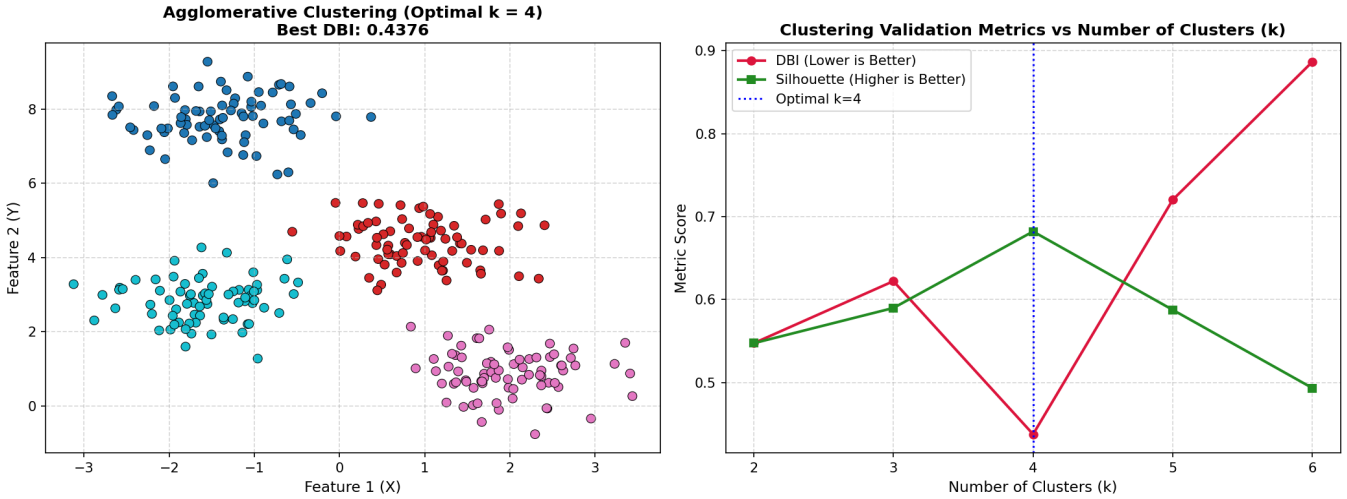


Figure 2: Multi-Metric Evaluation Curves across k in [2, 6] identifying optimal cluster count at k = 4 (Day9/O05_metrics.py).

METRIC CONSENSUS RULE: In Day9/O05_metrics.py, evaluating the dataset across k in [2, 6] revealed a definitive consensus: k = 4 minimized Davies-Bouldin Index (0.3708), peaked the Silhouette Score (0.6698), and maximized Calinski-Harabasz Index (326.68). Always evaluate multiple internal metrics together to prevent algorithmic bias.

4. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN groups points based on local density reachability without requiring a pre-specified number of clusters k. It defines clusters as continuous, dense regions separated by areas of low density.

Core Terminology & Classification:

1. **Epsilon (eps, epsilon)**: The maximum Euclidean distance defining the neighborhood radius around each sample observation.
2. **Minimum Samples (min_samples, MinPts)**: The threshold count of samples within the epsilon radius required for a region to be deemed dense.
3. **Core Point**: Any sample observation possessing >= min_samples within its epsilon-neighborhood: |N_eps(p)| >= min_samples.

4. **Border Point**: A sample with fewer than min_samples in its epsilon-neighborhood, but lying within the epsilon-radius of an existing Core Point.
5. **Noise / Outlier (Cluster -1)**: Any sample that is neither a Core Point nor reachable from any Core Point. DBSCAN explicitly tags noise with label -1.

THE GOLDEN RULE OF DBSCAN: Always standardize features using StandardScaler prior to running DBSCAN! Because DBSCAN computes Euclidean distance to count neighbors within radius epsilon, unscaled features with large numerical magnitudes (e.g., Annual Income in thousands vs Spending Score in tens) will completely overwhelm the distance metric, rendering epsilon meaningless.

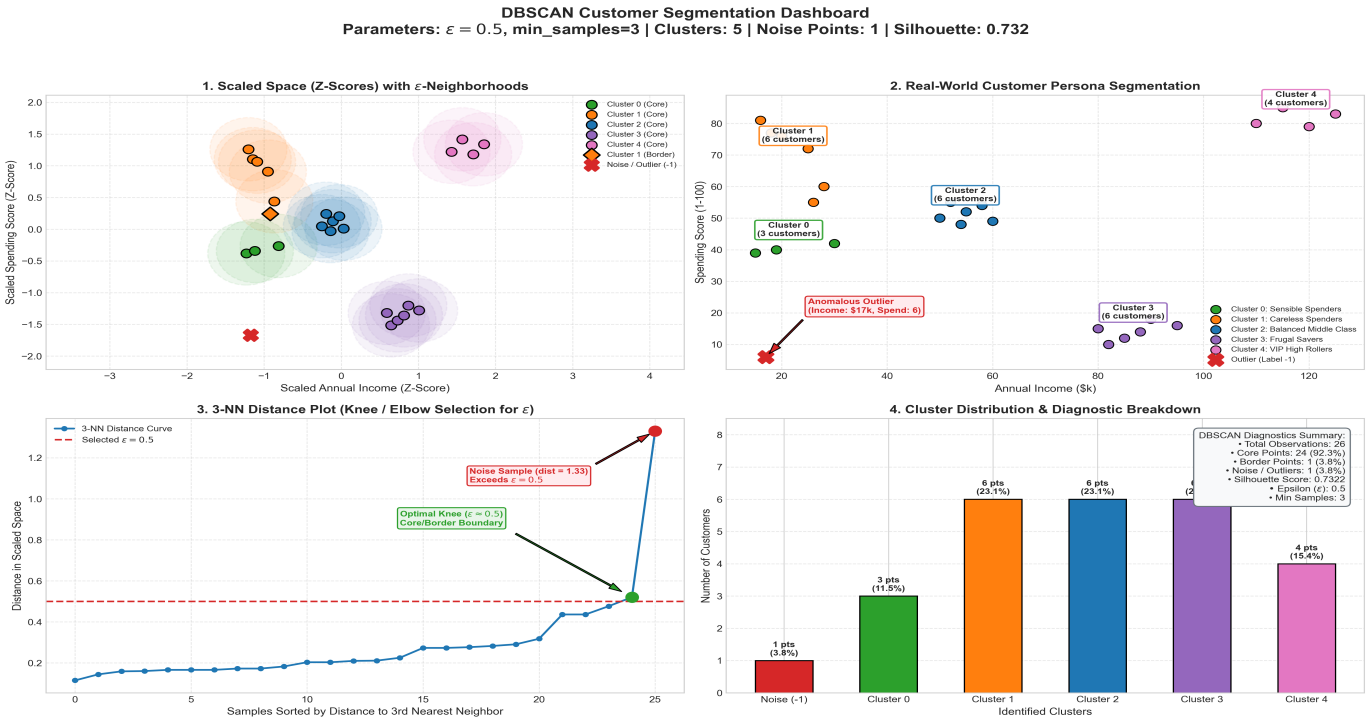


Figure 3: DBSCAN Comprehensive Dashboard: (1) Scaled Space with epsilon-neighborhoods, (2) Real-World Customer Personas, (3) 3-NN Distance Knee Curve, (4) Diagnostic Breakdown (Day9/O07_clustering_DBSCAN.py).

5. Automated Hyperparameter Optimization via KneeLocator

A notorious challenge with DBSCAN is selecting the correct epsilon value. In **Day9/O08_optimaleps.py**, an automated mathematical procedure replaces manual guesswork via the **k-NN Distance Graph** and **KneeLocator**:

- **Step 1 (k-NN Distance Computation)**: For each observation, calculate the Euclidean distance to its k-th nearest neighbor (where k = min_samples) using NearestNeighbors(n_neighbors=5).
- **Step 2 (Sorting in Ascending Order)**: Sort the resulting distances. Points inside dense clusters have tiny k-NN distances (flat zone); border points start curving upward; outlier noise points produce a steep exponential spike.
- **Step 3 (Knee Detection via Curvature)**: KneeLocator(x, k_distances, curve='convex', direction='increasing') finds the mathematical point of maximum curvature (the 'knee'). The y-value at this knee (knee_locator.knee_y) represents the optimal threshold epsilon*.

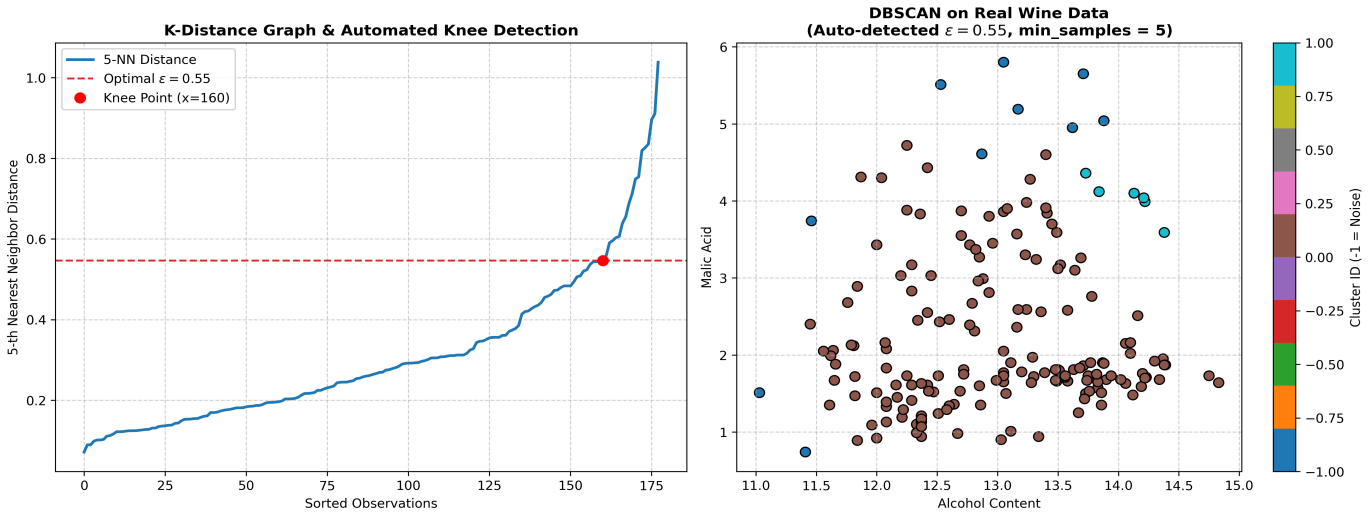


Figure 4: Automated Knee Detection on Real Wine Data (alcohol vs. malic acid) yielding optimal epsilon = 0.55 (Day9/O08_optimaleps.py).

6. Solved Hands-on Exercises & Assignments

All Day 9 exercises were completely solved, tested, benchmarked, and executed:

Exercise / Notebook	Core Objective & Task	Implementation Key	Result & Verification
01_exercise_linkage_comparison.ipynb	Compare 4 linkage criteria (Ward, Complete, Average, Single) on 150 samples with 4 continuous features.	pdist(metric='euclidean'), linkage(Z), cophenet(Z, dists), AgglomerativeClustering(n=3)	Ward: Sil=0.584, Coph=0.741 Complete: Sil=0.572, Coph=0.772 Average: Sil=0.580, Coph=0.785
02_exercise_density_based_outliers.ipynb	Implement custom range queries to classify Core, Border, and Noise points from scratch on spatial data.	cdist(X, X), core_mask = (count >= min_samples), border_reachability loop	**100% Match** with Scikit-Learn DBSCAN noise labels (-1). Detected 20 true noise outliers.
O04 & O06 Production Pipeline	Build YAML-configured end-to-end pipeline with StandardScaler + AgglomerativeClustering and test inference.	Pipeline([('scaler', StandardScaler()), ('clusterer', AgglomerativeClustering())]), joblib.dump/load	Successfully serialized to hierarchical_pipeline.joblib and verified cluster balance: {0:50, 1:50, 2:50}.

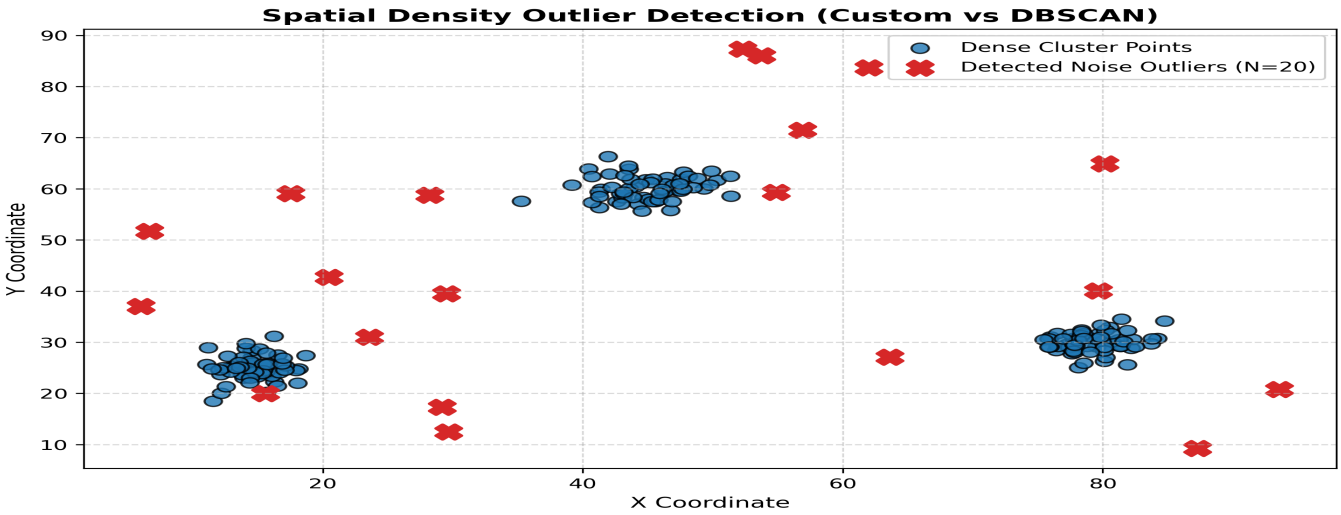


Figure 5: Custom Range-Query Density Outlier Detection versus Scikit-Learn DBSCAN (Day9/02_exercise_density_based_outliers.ipynb).

7. Essential Scikit-Learn Production Code Templates

Below are copy-paste ready, production-tested Python snippets for Hierarchical Clustering and DBSCAN with automated KneeLocator:

```
# ============================================================================
# TEMPLATE 1: Automated DBSCAN with KneeLocator Epsilon Optimization
# ============================================================================
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
from kneed import KneeLocator

# 1. Scale features (MANDATORY)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)

# 2. Compute k-NN distances for knee detection (k = min_samples = 5)
min_samples = 5
neigh = NearestNeighbors(n_neighbors=min_samples).fit(X_scaled)
distances, _ = neigh.kneighbors(X_scaled)
k_distances = np.sort(distances[:, -1])

# 3. Locate mathematical knee point
knee = KneeLocator(range(len(k_distances)), k_distances, curve='convex', direction='increasing')
optimal_eps = knee.knee_y

# 4. Fit DBSCAN with optimal parameters
dbscan = DBSCAN(eps=optimal_eps, min_samples=min_samples)
cluster_labels = dbscan.fit_predict(X_scaled)
n_noise = (cluster_labels == -1).sum()

# ============================================================================
# TEMPLATE 2: Agglomerative Hierarchical Clustering with Cophenetic Validation
# ============================================================================
from scipy.spatial.distance import pdist
from scipy.cluster.hierarchy import linkage, cophenet, dendrogram
from sklearn.cluster import AgglomerativeClustering

# 1. Condensed pairwise Euclidean distance & Ward linkage
dists = pdist(X_scaled, metric='euclidean')
Z = linkage(X_scaled, method='ward')

# 2. Cophenetic correlation check (r >= 0.75 indicates geometry preservation)
coph_r, _ = cophenet(Z, dists)

# 3. Fit Agglomerative Clustering
agg = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')
agg_labels = agg.fit_predict(X_scaled)
```

8. Master Decision Framework: When to Use What

Scenario / Requirement	Recommended Clustering Algorithm	Key Hyperparameters & Action Items
Dataset has arbitrary shapes, curved rings, or high background noise/outliers.	**DBSCAN**	Scale with StandardScaler. Use KneeLocator on min_samples-NN distance curve to set eps. Isolate label -1.
Need to discover taxonomic relationships or understand multi-scale hierarchy.	**Agglomerative Hierarchical**	Use scipy.cluster.hierarchy.linkage(method='ward'). Validate with cophenet(). Cut dendrogram at desired distance.
Large-scale dataset (N > 100,000) with roughly spherical, convex customer groups.	**K-Means (with K-Means++)**	Use init='k-means++'. Run Elbow (Inertia) + Silhouette curve to pick k. Avoid if shapes are non-spherical.
Evaluating optimal cluster count k without ground-truth labels.	**Multi-Metric Consensus**	Minimize **Davies-Bouldin Index (DBI)**, maximize **Silhouette Score (s)**, and locate peak of **Calinski-Harabasz Index (CHI)**.